OE-LLM: Orthogonal ELM-based large language model for fine-tuning acceleration

Qing Wang[a,*]

[a] *Southeast University, Nanjing 210018, China*

Abstract

Transformer-based large language models (LLMs) have rapidly grown in size and capability, but this progress comes with sharply increasing computational and memory demands *during fine-tuning*. In many practical deployments, especially on resource-constrained or shared hardware, the objective is to preserve a large fraction of baseline performance while substantially reducing the time and compute required for task adaptation.

A key bottleneck in fine-tuning is the feed-forward network (FFN) sublayer in Transformer blocks: FFNs account for a large share of trainable parameters and optimizer states, and their weight updates often dominate the end-to-end fine-tuning cost. Prior layer-wise analyses and probing studies consistently show that lower and middle layers capture more general linguistic features, while upper layers become increasingly task-aligned. This motivates a simple design principle: reduce trainability in the upper stack while preserving stable feature transformations.

Based on this principle, we propose OE-LLM, which replaces the trainable FFNs in the upper half of the encoder with a parameter-free orthogonal module inspired by *Extreme Learning Machine AutoEncoder* (ELM-AE). The core motivation is that orthogonal projections approximately preserve vector norms and angles, which stabilizes representation geometry when using fixed random features. As a result, OE-LLM keeps attention, residual connections and normalization intact, while removing upper-layer FFN gradients and the associated optimizer states.

Experiments on two Transformer-based language model backbones (BERT and RoBERTa) across IMDB, AGNews, MNLI, and XNLI show that OE-LLM retains more than 95% of the baseline accuracy on average while reducing total fine-tuning wall-clock time by 40–50%. The gains are consistently observed across datasets with different sequence lengths, and are accompanied by a substantial reduction in peak GPU memory.

Overall, OE-LLM demonstrates that replacing upper-layer FFN updates with an orthogonal ELM-AE-style fixed mapping provides a practical efficiency–accuracy trade-off for fine-tuning. The results suggest that significant fine-tuning acceleration can be achieved without modifying the attention mechanism or changing model depth, making the approach compatible with common system-level optimizations and efficient-attention kernels.

Keywords: Transformer, feed-forward network, Extreme Learning Machine AutoEncoder, orthogonal projection, fine-tuning acceleration

1. Introduction

Transformer-based language models have evolved from task-specific architectures into general-purpose backbones that support a wide range of NLP applications. As model sizes and dataset scales have grown, so have the computational and memory costs of adapting these models to new tasks. Even for standard configurations, fine-tuning can be a dominant consumer of GPU time and energy, especially when many tasks, domains or user-specific variants need to be maintained in parallel. In constrained environments such as edge devices and small embedded platforms, this cost can quickly become the main obstacle to deploying Transformer-based intelligence at scale [1].

A substantial portion of this cost comes from the position-wise feed-forward networks (FFNs) that appear in every Transformer block. During fine-tuning, these FFNs contribute not only to the forward computation but also to a large fraction of the gradient and optimizer updates. When models are scaled up or when many fine-tuning jobs run concurrently, the cumulative cost of these FFN updates can become a practical bottleneck, even if peak accuracy is already satisfactory and further gains would have diminishing returns in downstream applications.

A broad set of techniques has been proposed to reduce the overall cost of Transformer models. Pruning methods remove attention heads, neurons or entire blocks that appear redundant for a given task [2, 3]. Quantization compresses parameters and sometimes optimizer states into lower-precision formats to save memory and arithmetic [4, 5]. Other work focuses on the attention mechanism itself, using sparse or linear attention and long-context designs to ease the quadratic dependence on sequence length [6–8]. Parameter-efficient tuning methods add small task-specific components, such as adapters, prompts or low-

*Corresponding author
Email address: wangqing17@seu.edu.cn (Qing Wang)

rank matrices, on top of largely frozen backbones [9, 10]. Choices in positional encoding, normalization and optimization further influence the stability and throughput of large-scale fine-tuning [11–14]. These directions have proven effective, but often demand additional implementation effort, custom kernels or extra training stages.

In this context, a different, more localized strategy is explored that aims to reduce the cost of FFNs in upper layers while leaving the rest of the architecture unchanged. The design is inspired by Extreme Learning Machines (ELMs) and ELM-based autoencoders, which use fixed random hidden mappings together with learned output layers [15, 16]. This idea is adapted to Transformer FFNs by replacing, in the upper half of layers, the two trainable FFN projections with a fixed orthogonal module composed of a matrix Q, the baseline nonlinearity and its transpose $Q^\top$. The matrix Q is initialized at random, orthogonalized by QR or SVD, and then frozen. As a consequence, these FFN sublayers no longer have trainable weights; their gradients and optimizer states are removed, while residual connections and normalization remain exactly as in the original model.

This modification is instantiated on BERT-base and RoBERTa-base and evaluated on four benchmarks: IMDB, AGNews, MNLI and XNLI. Across these tasks, the modified models retain on average more than 95% of the baseline accuracy and reduce total fine-tuning wall-clock time (training plus evaluation) by 40–50% on three datasets, with smaller yet consistent improvements for longer inputs. The largest accuracy reductions are observed on NLI benchmarks, typically around three percentage points, whereas single-sentence classification is less affected. These trends are consistent with layer-wise analyses indicating that lower and middle layers primarily encode lexical and syntactic information, while upper layers are more tightly linked to task-specific decision surfaces [17, 18]. In many practical scenarios, including edge and small-device deployment, such a trade-off—maintaining the vast majority of the accuracy while effectively halving fine-tuning cost—is preferable to marginal improvements in peak performance.

The proposed change is confined to the FFN sublayers, does not introduce new trainable parameters and does not alter attention, depth or normalization. As such, it can be integrated into existing implementations with minimal code changes and combined with many of the acceleration and compression techniques developed in parallel.

Contributions.. This work makes three main contributions.

- **A minimal FFN-side modification for fine-tuning acceleration.** We propose OE-LLM, which replaces the trainable FFNs in the upper half of Transformer layers with a parameter-free orthogonal ELM-style mapping of the form $Q^\top \sigma(Qx)$. The modification leaves attention, depth, residual connections, and normalization unchanged, introduces no additional trainable parameters, and can be implemented with minimal code changes in standard Transformer backbones.
- **End-to-end measurement of fine-tuning efficiency under matched protocols.** We report wall-clock training and evaluation time together with peak GPU memory under a matched fine-tuning setup (fixed max sequence length, batch sizes, epochs, and random seeds). This provides a practical characterization of user-visible acceleration beyond FLOPs or parameter counts.
- **Scientific findings on the operating regime of fixed orthogonal FFNs.** Beyond aggregate accuracy and time gains, we conduct systematic comparisons and ablations to characterize the operating regime of the proposed replacement. We find that orthogonal projections help stabilize fixed upper-layer mappings, that replacement depth and layer location largely determine the efficiency–accuracy trade-off, and that most remaining degradation concentrates on NLI tasks. These results explain the method's behavior and guide practical layer-selection choices.

2. Related Work

2.1. Encoder architectures and layer roles

Transformer encoders combine self-attention with position-wise FFNs in each block [19]. Pretrained models such as BERT and RoBERTa follow this pattern and achieve strong results on many NLP benchmarks [20, 21]. Analyses of internal representations show that different layers tend to specialize. Lower and middle layers capture lexical and syntactic information, while upper layers are more tightly aligned with the downstream objective [17, 18]. Similar conclusions have been drawn from layer-wise probing on QA and NLI, where the top layers are most important for final predictions [22]. These findings suggest that it may be more forgiving to restrict capacity in upper layers than in lower ones, provided that earlier layers remain flexible.

2.2. Acceleration and compression

A number of methods accelerate or compress Transformer models. Pruning removes parts of the network—such as attention heads or hidden units—that contribute little to task performance [2, 3]. Distillation trains smaller students to mimic larger teachers and often yields compact models with strong accuracy [23, 24]. Quantization reduces memory and compute by representing parameters and sometimes optimizer states with fewer bits [4, 5].

Another line of work focuses on attention and memory. Sparse and linear attention formulations reduce the quadratic dependence on sequence length by restricting or approximating token interactions [6, 7]. Long-context architectures extend the range of dependencies while controlling cost on long documents or sequences [8, 25]. At

the system level, techniques such as ZeRO partition optimizer states across devices to enable training larger models within memory limits [26].

Parameter-efficient tuning (PEFT) offers yet another perspective. Adapters and prompt-based methods add relatively small modules or embeddings while freezing most backbone weights [9, 27]. Low-rank update schemes such as LoRA attach compact trainable matrices to existing projections [10, 28]. These approaches significantly reduce the number of trainable parameters per task but do not directly change the computational cost of the backbone FFNs.

The present method is complementary to these directions. It leaves attention and overall depth unchanged and does not add new trainable components. Instead, it removes FFN trainability in a chosen subset of layers by substituting a fixed orthogonal mapping.

2.3. Orthogonal mappings, random features and ELMs

Orthogonal or near-orthogonal weight matrices are often used to stabilize optimization in deep networks. They help maintain well-conditioned singular values and mitigate gradient explosion or vanishing through depth [29, 30]. Random feature methods project inputs onto fixed random directions and apply nonlinearities to approximate kernel mappings in a finite-dimensional space [31].

ELMs combine these ideas in a simple architecture: hidden-layer weights are fixed and random, while only the output weights are trained [15]. Variants such as ELM autoencoders extend this to unsupervised representation learning [16]. Later work investigates deeper ELM-style models and structured random features, including orthogonal constructions [32–34]. In particular, ELM-based dimension reduction uses orthogonal or approximately orthogonal random encoders together with learned linear decoders, and shows that such encoders can preserve distances and class structure in a lower-dimensional space [32].

This principle is adapted here to Transformer FFNs. The upper-layer FFNs are replaced by a fixed orthogonal module, so they act as random nonlinear feature mixers rather than fully trainable components. Lower FFNs and the classifier head remain trainable and compensate for the lost degrees of freedom in the upper layers.

3. Methodology

3.1. Overview

The objective is to reduce fine-tuning cost by modifying only the FFN sublayers in upper Transformer blocks. For an encoder with N layers, indexed by $i = 1, \ldots, N$, layers 1 to $N/2$ are kept unchanged and only the FFNs in layers with $i \geq N/2$ are altered. In each of these layers, the original FFN is replaced by a parameter-free module based on a fixed orthogonal matrix Q and its transpose $Q^\top$. Self-attention, residual connections and normalization are kept exactly as in the pretrained backbone.

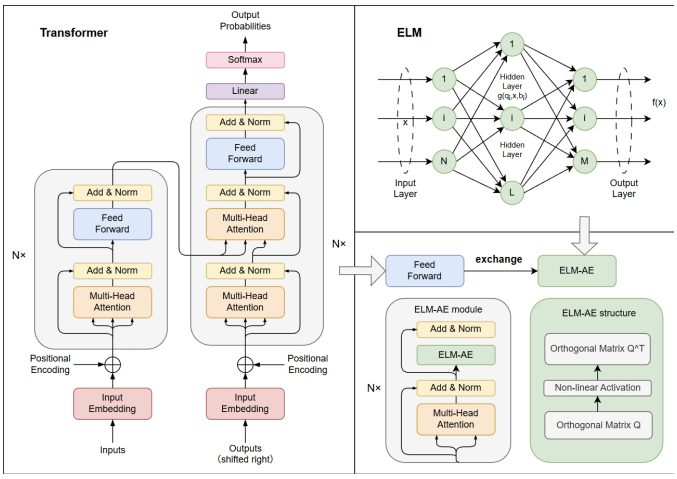

Figure 1: Overall architecture. The left panel illustrates a standard Transformer encoder with stacked attention and FFN blocks. The upper-right panel shows a classical ELM network with a fixed random hidden layer and a trainable linear output layer. The lower-right panel depicts the proposed combination, in which upper-layer Transformer FFNs are replaced by an ELM-AE-style module built from a fixed orthogonal mapping Q, a nonlinearity, and its transpose $Q^\top$, while the surrounding attention, residual and normalization structure remains unchanged.

3.2. Standard Transformer FFN

In a standard block, the position-wise FFN has the form

$$\mathrm{FFN}(x) = W_2\, \sigma(W_1 x + b_1) + b_2, \tag{1}$$

where $x \in \mathbb{R}^{d_{\text{model}}}$ is the input at a single position, $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ and $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ are trainable matrices, $b_1 \in \mathbb{R}^{d_{\text{ff}}}$ and $b_2 \in \mathbb{R}^{d_{\text{model}}}$ are biases, and σ is a pointwise nonlinearity such as GELU or a gated variant [35, 36]. The FFN output is combined with residual connections and a normalization layer such as LayerNorm or RMSNorm [12, 37].

Ignoring biases, the parameter count in one FFN sublayer is approximately

$$P_{\text{FFN}} \approx 2 d_{\text{ff}} d_{\text{model}}, \tag{2}$$

In typical base models with $d_{\text{ff}} \approx 4 d_{\text{model}}$, FFNs represent a substantial share of encoder parameters and hence of gradient and optimizer updates during fine-tuning.

3.3. Fixed orthogonal replacement

The learned FFN projections in upper layers are replaced with an ELM-style fixed mapping [15, 16]. The modified sublayer is

$$\text{ELM-FFN}(x) = Q^\top \sigma(Qx), \tag{3}$$

where $Q \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is a fixed orthogonal matrix. Concretely, Q is constructed with orthonormal columns by sampling a tall Gaussian matrix and applying QR decomposition, so that

$$Q^\top Q = I_{d_{\text{model}}}, \tag{4}$$

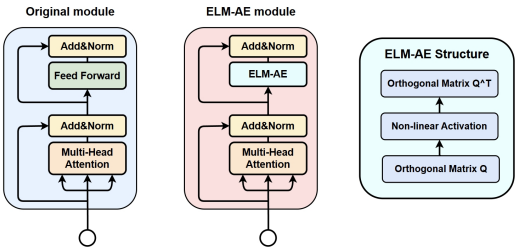

Figure 2: Schematic illustration of the standard FFN and the proposed ELM-FFN. The left panel shows the standard Transformer FFN with two trainable linear projections and a nonlinearity. The middle panel depicts an upper-layer block where the FFN sublayer is replaced by the ELM-FFN while attention, residual and normalization components are preserved. The right panel expands the ELM-FFN itself: the input representation flows from bottom to top through the fixed orthogonal up-projection Q, the pointwise nonlinearity σ, and the down-projection $Q^\top$.

This choice is consistent with the usual setting $d_{\text{ff}} > d_{\text{model}}$ and ensures that the up-projection approximately preserves norms in the model space. Biases are omitted and the activation σ is kept the same as in the original model.

For layers with $i \geq N/2$, this implies:

- The parameters (W_1, b_1) and (W_2, b_2) are removed and no longer appear in the optimizer.
- Q and $Q^\top$ are treated as non-trainable; gradients can flow through them to earlier layers, but the matrices themselves remain fixed.
- No extra loss terms or pretraining phases are introduced; standard fine-tuning is applied on top of the modified backbone.

To assess the importance of orthogonality, a variant is also considered where Q is drawn at random but not orthogonalized. This variant preserves the structure of $Q^\top\sigma(Qx)$ but no longer approximates an isometry. Section 4.6 reports a direct comparison of these two designs.

3.4. Random feature and layer-wise perspectives

The mapping $x \mapsto Q^\top\sigma(Qx)$ can be interpreted as a random feature expansion followed by a projection back to the model space. The columns of Q define random directions, and the coordinates $\sigma(q_j^\top x)$ act as nonlinear features. Aggregating these responses through $Q^\top$ yields a finite-dimensional approximation to a richer nonlinear transformation, in the spirit of random feature methods for kernels [31]. For a moderately large hidden dimension d_{ff}, this mapping can be expressive while remaining parameter-free.

From the perspective of Extreme Learning Machines (ELMs), this fixed FFN is a special case of an ELM-style hidden layer: Q and the nonlinearity σ define a random feature map $h(x) = \sigma(Qx)$, while the remaining trainable layers of the encoder and the classifier head collectively play the role of the output weights that adapt these features to the downstream task [15, 32]. ELM-based dimension reduction further shows that orthogonal or approximately orthogonal random encoders, combined with suitable linear decoders, can preserve distances and class structure in a compressed representation space [32]. Although the objective here is supervised prediction rather than reconstruction, these results support the idea that frozen random encoders with orthogonal structure can still provide useful and geometrically well-behaved feature spaces when coupled with trainable downstream layers.

A second perspective comes from empirical studies of Transformer layers. Probing and diagnostic work suggests that lower and middle layers primarily encode lexical and syntactic information, whereas upper layers are more closely aligned with the downstream decision surface [17, 18]. Under this view, preserving full trainability in lower FFNs maintains the ability to sculpt intermediate linguistic features. In the upper layers, one can afford to restrict FFNs to a fixed, near-isometric nonlinear mixing, provided that the classifier head and remaining trainable blocks are allowed to adapt.

3.5. Geometric intuition for orthogonal ELM-FFNs

It is instructive to consider why a fixed orthogonal module of the form

$$x \mapsto Q^\top\sigma(Qx), \tag{5}$$

can be a reasonable replacement for a trainable FFN from a geometric point of view. If the nonlinearity σ is temporarily ignored, the mapping $x \mapsto Qx$ with $Q^\top Q = I_{d_{\text{model}}}$ is an isometric embedding from $\mathbb{R}^{d_{\text{model}}}$ into $\mathbb{R}^{d_{\text{ff}}}$: for any $x_1, x_2 \in \mathbb{R}^{d_{\text{model}}}$,

$$\|Qx_1 - Qx_2\|_2^2 = (x_1 - x_2)^\top Q^\top Q(x_1 - x_2) = \|x_1 - x_2\|_2^2, \tag{6}$$

In other words, the linear part of the ELM-FFN preserves Euclidean distances and angles exactly in the absence of nonlinearities. This role of orthogonal random encoders is consistent with ELM-based dimension reduction, where orthogonal or approximately orthogonal input weights are used to construct mappings that preserve pairwise distances and class structure in a compressed space [32].

When a pointwise nonlinearity σ is inserted between Q and $Q^\top$, the exact isometry no longer holds, but the distortion can be controlled if σ is Lipschitz. For example, if σ is L-Lipschitz, then for any x_1, x_2,

$$\|\sigma(Qx_1) - \sigma(Qx_2)\|_2 \leq L\,\|Qx_1 - Qx_2\|_2 = L\,\|x_1 - x_2\|_2, \tag{7}$$

using (6). The subsequent projection $Q^\top$ then satisfies

$$\|Q^\top\sigma(Qx_1) - Q^\top\sigma(Qx_2)\|_2 \leq \|Q\|_2\,\|\sigma(Qx_1) - \sigma(Qx_2)\|_2, \tag{8}$$

and $\|Q\|_2 = 1$ because $Q^\top Q = I_{d_{\text{model}}}$. Overall, the ELM-FFN behaves as a structured, norm-controlled perturbation of the identity mapping in the upper layers.

From the ELM viewpoint, the transformation $h(x) = \sigma(Qx)$ defines a fixed random feature map, and the rest of the encoder together with the classifier head implements a trainable function of $h(x)$ [15, 31, 32]. Classical results on random features and ELMs show that, for sufficiently rich feature maps h and moderate width d_{ff}, such architectures can approximate a wide class of target functions with only the output-side weights being trained. In this setting, this suggests that freezing the upper-layer FFNs to an orthogonal ELM-style module mainly changes *where* in the network the task-specific adaptation occurs (lower layers and the classifier), rather than preventing the model from fitting the downstream tasks.

3.6. Effect on computation and memory

Forward FLOPs for the original FFN and the ELM-FFN are nearly identical: both apply two dense projections and a nonlinearity per token. The main savings arise in the backward pass and optimizer:

- **Backpropagation.** In the baseline, gradients are computed for W_1, W_2, b_1 and b_2 in each FFN layer. When these parameters are removed in upper layers, their gradient computation is no longer needed. Gradients still flow through Q and $Q^\top$ but do not update them.
- **Optimizer state.** Adaptive methods such as AdamW or Adafactor maintain auxiliary statistics (e.g., first and second moments or factored statistics) for each trainable parameter [13, 14, 38]. Removing upper-layer FFN weights directly reduces the amount of optimizer state and the per-step update cost.

The net wall-clock speedup thus depends on how much FFNs contribute to the overall budget relative to attention, embeddings and evaluation overheads. On shorter sequences and moderate batch sizes, FFNs take a sizeable fraction of total time, so removing half of their weight updates yields substantial gains. On long-document inputs, where attention and evaluation dominate, the improvement is smaller but still positive [8, 25]. In all cases, the aim is not to extract the last fraction of a percentage point in accuracy, but to achieve a stable operating point where accuracy remains high and the compute footprint is significantly reduced, which is particularly valuable under tight resource budgets.

3.7. Design choices

Two design choices are particularly important in this setting:

Which layers to modify.. A simple cut at half depth is used: layers 1 to $N/2$ are left unchanged, and layers $N/2+1$ to N use ELM-FFNs. This choice reflects the layer-wise observations mentioned above and keeps the design easy to implement. In Section 4.6, experiments with shallower and deeper cuts are also reported to gauge sensitivity.

Activation and width.. The original activation function and FFN width d_{ff} of the backbone are kept unchanged. This ensures that any difference in behavior is due to the replacement of learned weights by a fixed orthogonal mapping, rather than to a change in hidden dimension or nonlinearity. Preliminary experiments with narrower ELM-FFNs suggest that substantial width reduction leads to additional accuracy loss, so aggressive shrinking of d_{ff} is not pursued in this work.

4. Experiments

4.1. Models and overall protocol

The proposed FFN replacement is instantiated on two widely used Transformer backbones: BERT-base [20] and RoBERTa-base [21]. Both have 12 encoder layers with hidden size $d_{\text{model}} = 768$ and FFN size $d_{\text{ff}} = 3072$. In the default configuration, the FFNs in the upper six layers ($i \geq 7$) are replaced with the ELM-FFN module described in Section 3. All other components, including embeddings, attention, residual connections and normalization, remain unchanged.

For each backbone, the following variants are compared:

- **Baseline**: the original encoder with fully trainable FFNs in all layers.
- **ELM-AE**: the encoder with ELM-FFN replacements in the upper half of layers.

Both variants share the same tokenizer, training schedule and optimization hyperparameters for a given dataset.

4.2. Datasets and tasks

Evaluation is conducted on four benchmarks that differ in input length and task structure:

- **IMDB** [39]: binary sentiment classification on movie reviews. Inputs are long user-generated texts; after subword tokenization many examples contain several hundred tokens. This setup increases attention and evaluation cost.
- **AGNews** [40]: four-way news topic classification with short headlines and brief descriptions. Sequences are much shorter than in IMDB, so FFNs account for a relatively larger share of per-step computation.
- **MNLI** [41]: a multi-genre natural language inference benchmark. Each example consists of a premise and a hypothesis, and the task is to predict entailment, contradiction or neutrality. The dataset spans several genres and provides matched and mismatched evaluation splits.

- **XNLI** [42]: a multilingual extension of MNLI constructed by translating a subset of examples. The English portion is used to align with monolingual pretraining. Compared with MNLI, XNLI is smaller and has a slightly different distribution, offering a complementary NLI evaluation.

These tasks jointly cover single-sentence and sentence-pair settings, as well as short and long inputs. They therefore provide a useful testbed for studying how the proposed modification interacts with both FFN cost and attention cost in regimes that are relevant for realistic deployment.

4.3. Hardware and timing setup

All experiments are conducted using Python 3.9, PyTorch 2.0 with CUDA 12.1, and HuggingFace Transformers 4.30.

Main experiments (Figure 3).. Unless stated otherwise, the main experiments used to produce the headline Baseline vs. ELM-AE comparisons and the figure in Section 5 are run on a single NVIDIA GeForce RTX 3090 GPU (24 GB VRAM), with 4 data-loader workers. For these runs, we use per-device batch size 8 for training and 16 for evaluation, and set the maximum sequence length to 512 for all datasets. Gradient accumulation, `torch.compile`, and mixed-precision training are not used in the reported timing results. Wall-clock time is measured with `time.time()` around the training and evaluation phases and excludes dataset preprocessing/tokenization and model initialization.

Supplementary baseline comparisons (Tables 1–2).. To contextualize ELM-AE against representative alternatives (Freeze-top and LoRA), we additionally run a four-way comparison (Baseline/ELM-AE/Freeze-top/LoRA) on a separate workstation equipped with an NVIDIA RTX Pro 6000 GPU. These supplementary runs follow the same training protocol (batch sizes, maximum sequence length, number of epochs, and evaluation procedure). *Importantly, absolute wall-clock times are hardware-dependent and are therefore* ***not*** *compared across the RTX 3090 and RTX Pro 6000.* Cross-machine analysis is restricted to checking whether the *relative* trends (e.g., time reductions relative to the Baseline measured on the *same* machine) remain consistent.

4.4. Training and evaluation setup

All models are implemented using HuggingFace Transformers [43] and PyTorch [44]. Orthogonal matrices (Q and $Q^\top$) are created once per modified layer by drawing a random Gaussian matrix and applying QR decomposition. The resulting Q is stored as a non-trainable tensor; $Q^\top$ is obtained by transposition. **To avoid ambiguity, we emphasize that Q is resampled independently for each run (random seed): in each run we generate a fresh Gaussian matrix, orthogonalize it, and then keep it fixed (not updated by gradients) throughout fine-tuning.** Consequently, the reported mean and variance over seeds reflect not only data shuffling and classifier-head initialization, but also the randomness induced by sampling Q.

Fine-tuning uses minibatch optimization with AdamW or Adafactor [14, 38]. Learning rates, batch sizes and training steps follow common BERT/RoBERTa practice for each dataset, and are kept identical for Baseline and ELM-AE under a matched protocol. Efficient attention kernels (SDPA or FlashAttention/FlashAttention-2) are optionally enabled for both variants when supported by the hardware and software stack [45, 46]. Activation checkpointing is not used in the reported results.

For each (model, dataset) pair, training is performed for 3 epochs and followed by a single final evaluation (i.e., no intermediate evaluation during training). Ten different random seeds are used per configuration; seeds affect data shuffling and parameter initialization of the classification head.

4.5. Metrics and reporting

For all four benchmarks test accuracy is reported as the primary performance metric. Two additional quantities are also reported:

- **Total fine-tuning wall-clock time**: elapsed time from the beginning of training to the end of the final evaluation, measured as described above.
- **Relative speedup**: the percentage reduction in wall-clock time for ELM-AE compared with the Baseline under identical settings.

All results are averaged over 10 seeds. For the main experiments, we report mean and standard deviation in Figure 3. For supplementary comparisons reported in tabular form, we report mean values under the same protocol.

4.6. Ablation studies

To better understand the behavior of the proposed modification, three sets of ablation-style experiments are conducted. One focuses on orthogonality and is reported quantitatively; the others examine, in a more exploratory way, how the number and location of replaced layers affect the trade-off between speed and accuracy.

Orthogonal vs. random replacements.. The first ablation directly compares the ELM-FFN module with an otherwise identical variant that uses random but non-orthogonal matrices in the upper-half FFNs. In the random variant, Q is sampled from a standard Gaussian distribution and used without orthogonalization. Table 3 in Section 5 shows that the orthogonal construction consistently outperforms the non-orthogonal one, especially on NLI tasks. This supports the view that approximate norm preservation in Q and $Q^\top$ is important for maintaining useful geometry in the upper-layer representations.

Depth of replacement.. The second ablation varies how many layers use ELM-FFNs. Besides the default choice of replacing the upper six layers in a 12-layer encoder, shallower and deeper cuts are considered, for example replacing only the top four layers or all but the bottom two layers. Qualitatively, the following trends are observed:

- Replacing fewer upper layers yields smaller time savings but also smaller accuracy reductions.
- Replacing more layers further reduces wall-clock time but leads to noticeable accuracy degradation, particularly on MNLI and XNLI.

These experiments are exploratory rather than exhaustive, but the trends align with the expectation that progressively restricting trainable capacity further down the stack makes it harder for the model to adapt to complex tasks.

Location of replacement.. Finally, the replacement window is shifted while keeping the number of modified layers fixed. For example, replacing layers 5 to 10 instead of 7 to 12 moves the frozen FFNs closer to the middle of the network. In these preliminary tests, cuts closer to the top (the default choice) achieve a more favorable balance between speed and accuracy than cuts that extend further into the middle layers. This observation is again consistent with layer-wise analyses that attribute more task-specific behavior to the uppermost layers.

These ablations suggest that orthogonality, the number of replaced layers and their location all influence the outcome. In the remainder of the paper, the focus is on the default setting of replacing the upper half of layers with orthogonal ELM-FFNs, which offers a simple and robust compromise between efficiency and performance in regimes where compute and memory are at a premium.

5. Results and Analysis

Figures 3 summarize, for each dataset, the mean test accuracy (top) and the *total fine-tuning wall-clock time* (bottom; training plus evaluation) for the Baseline and ELM-AE variants. The speedup is defined as

$$\text{Speedup} = 100 \times \left(1 - \frac{\text{Time}_{\text{method}}}{\text{Time}_{\text{Baseline}}}\right).$$

5.1. Performance comparison

To contextualize the proposed replacement relative to common alternatives, we additionally compare against two widely used baselines: (i) a *freeze-top* strategy that freezes the upper half of encoder layers and trains only the lower half and the classifier head, and (ii) a representative PEFT setup based on LoRA applied to attention query/value projections. *These four-way comparisons are conducted on a separate GPU platform (RTX Pro 6000) and are reported as supplementary results. Absolute wall-clock times*

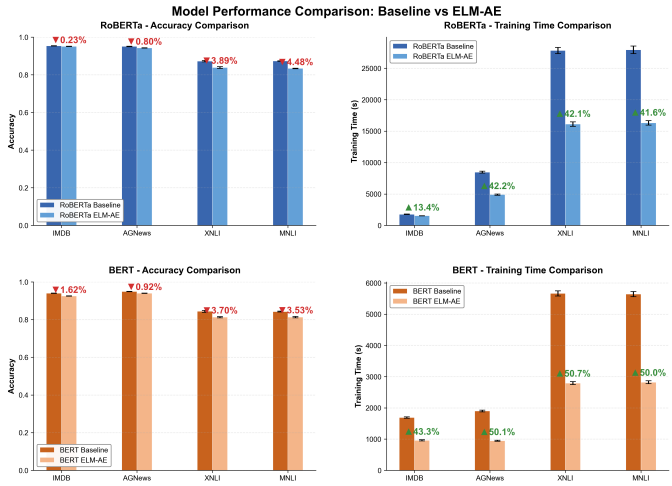

Figure 3: RoBERTa (upper panels) and BERT (lower panels): accuracy (left of each pair) and total fine-tuning time (right of each pair, seconds; training + evaluation) on IMDB, AGNews, XNLI and MNLI. Bars show mean ± standard deviation over seeds for Baseline and ELM-AE; time panels annotate the average speedup for ELM-AE.

in Tables 1–2 are therefore ***not*** *directly comparable to the seconds shown in Fig. 3, which are measured on RTX 3090.* Table 1 reports mean accuracy and F1 across the four datasets under a matched training protocol.

Several patterns are apparent. First, ELM-AE remains close to the full fine-tuning baseline on single-sentence classification, with modest drops on IMDB (0.9412 → 0.9268) and AGNews (0.9434 → 0.9395). Second, the gap widens on NLI, where restricting the upper-layer FFNs leads to larger degradation (e.g., 0.8401 → 0.8167 on MNLI and 0.8474 → 0.8161 on XNLI). Third, the two additional baselines illustrate that fewer trainable parameters do not automatically translate into better accuracy: while LoRA trains only a tiny fraction of parameters, it shows a noticeably larger drop on MNLI/XNLI in this matched setting.

5.2. Fine-tuning time reductions

Table 2 reports a breakdown of training time, final evaluation time, and total fine-tuning time. **All times in this table are measured on RTX Pro 6000 and are not directly comparable (in absolute seconds) to Fig. 3, which is measured on RTX 3090.** In addition, it lists peak GPU memory and the fraction of trainable parameters (relative to the full fine-tuning baseline). Because the protocol performs a single evaluation after training, the evaluation component is small for AGNews, MNLI, and XNLI, whereas it is more visible for IMDB due to longer sequences.

Two observations stand out from this breakdown. First, ELM-AE yields the largest reduction in *training* time across all datasets, translating into substantial end-to-end speedups on this platform: from 41.6% on AGNews up to 52.7% on MNLI, while also reducing peak memory on this platform from 3.11–3.81 GB (Baseline) to 1.25–1.50 GB (ELM-

Table 1: BERT-base: mean test accuracy and F1 for four fine-tuning strategies under matched settings (single GPU, FP32, max length 512, 3 epochs, and a single final evaluation). **Measured on RTX Pro 6000.** Accuracy is hardware-independent; wall-clock times for this platform are reported separately in Table 2.

	Baseline		ELM-AE		Freeze-top		LoRA	
Dataset	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
IMDB	0.9412	0.9412	0.9268	0.9268	0.9370	0.9370	0.9133	0.9133
AGNews	0.9434	0.9434	0.9395	0.9394	0.9486	0.9486	0.9188	0.9187
MNLI	0.8401	0.8409	0.8167	0.8171	0.8309	0.8312	0.7767	0.7773
XNLI	0.8474	0.8480	0.8161	0.8164	0.8293	0.8295	0.7671	0.7684

Table 2: BERT-base: time breakdown and resource footprint for four strategies (mean values). **Measured on RTX Pro 6000. Absolute times are hardware-dependent and are therefore not directly comparable to Fig. 3 (RTX 3090).** Speedup is computed from total wall-clock time relative to Baseline *on the same machine*. "Params" denotes the fraction of trainable parameters relative to Baseline.

Dataset	Method	Train	Eval	Total	Speedup (%)	Mem (GB)	Params (%)
IMDB	Baseline	668.3	96.0	764.3	0.0	3.81	100.00
	ELM-AE	352.7	59.5	412.2	46.1	1.50	13.50
	Freeze-top	503.1	59.5	562.7	26.4	2.86	39.38
	LoRA	457.1	62.2	519.3	32.1	2.47	0.27
AGNews	Baseline	1643.5	5.2	1648.7	0.0	3.11	100.00
	ELM-AE	958.2	4.6	962.8	41.6	1.25	13.51
	Freeze-top	1519.9	9.4	1529.2	7.2	2.30	39.39
	LoRA	1255.4	5.4	1260.8	23.5	1.93	0.27
MNLI	Baseline	4845.5	5.4	4850.9	0.0	3.45	100.00
	ELM-AE	2288.4	5.5	2293.9	52.7	1.37	13.50
	Freeze-top	3330.7	5.6	3336.3	31.2	2.58	39.38
	LoRA	4296.8	6.0	4302.8	11.3	2.21	0.27
XNLI	Baseline	4101.5	1.2	4102.7	0.0	3.45	100.00
	ELM-AE	2315.7	1.1	2316.8	43.5	1.37	13.50
	Freeze-top	3377.6	1.2	3378.8	17.7	2.58	39.38
	LoRA	3675.7	1.5	3677.2	10.4	2.21	0.27

AE). Second, the comparison with Freeze-top and LoRA highlights that the mapping between trainable parameter count and wall-clock time is not monotonic. Freeze-top retains a nontrivial fraction of trainable parameters and therefore cuts training time only moderately, while LoRA trains very few parameters but still pays the cost of full forward/backward through the backbone, leading to smaller or inconsistent speedups depending on the dataset.

5.3. Trade-offs between time and accuracy

Taken together, Tables 1 and 2 suggest a clear operating regime for ELM-AE. Relative to full fine-tuning, it delivers a large reduction in wall-clock time and memory footprint, with small accuracy loss on IMDB and AGNews and a more pronounced drop on NLI. Compared with simple freezing, ELM-AE achieves markedly larger speedups on all four datasets. Compared with LoRA in this matched setup, ELM-AE provides higher accuracy on NLI and substantially larger speedups, indicating that removing upper-layer FFN updates targets a different bottleneck than purely parameter-efficient tuning.

5.4. Ablation: orthogonal vs. non-orthogonal matrices

To isolate the effect of orthogonality, the ELM-AE module is compared with a variant that uses random but non-orthogonal matrices in the upper-half FFNs. All other components and hyperparameters are kept the same. Table 3 reports BERT test accuracy on the four datasets.

Table 3: Ablation on BERT: orthogonal ELM-AE versus non-orthogonal random matrices (test accuracy).

Dataset	ELM-AE (ortho)	Random (non-ortho)
IMDB	0.9247	0.9228
AGNews	0.9403	0.9397
XNLI	0.8162	0.8095
MNLI	0.8148	0.8052

The orthogonal construction consistently outperforms the non-orthogonal one. The margin is small but positive on IMDB and AGNews, and more pronounced on XNLI and MNLI. This supports the idea that approximate norm preservation in the fixed projections helps maintain useful geometry in upper-layer representations and stabilizes optimization, especially for tasks that require fine-grained semantic distinctions [29, 30].

5.5. Relation to other acceleration methods

Compared with pruning, distillation and quantization, the proposed method keeps the encoder depth and hidden sizes unchanged but removes FFN trainability in selected layers [2, 23]. Quantization mainly targets precision and memory, while efficient attention redefines how tokens interact in sequence [6, 7]. ELM-FFNs do not modify at-

tention or embedding structure; they instead remove FFN gradients and optimizer states in the upper half of the encoder.

Because of this separation of concerns, the approach can be combined with many of the above techniques. For example, attention kernels and ZeRO-style optimizers can reduce memory and attention cost [26, 45], while ELM-FFNs further shrink the training cost associated with upper-layer FFNs. For deployment on edge hardware or shared accelerators, this combination offers a concrete recipe: first apply system- and attention-level optimizations, then reduce FFN-side training cost via fixed orthogonal replacements.

5.6. Interpretation

Two factors help explain why accuracy remains high on most tasks. First, orthogonal mappings preserve norms and angles to a large extent, which stabilizes signal propagation and avoids extreme distortions across depth [29]. Second, the residual and normalization structure of the encoder is unchanged [12]. Taken together with the layer-wise specialization picture [18], this suggests that most of the burden of adaptation can be shifted to lower FFNs and the classifier head, as long as upper FFNs still provide a rich but fixed nonlinear mixing.

The more noticeable degradation on NLI can then be understood as the cost of constraining exactly those layers that carry much of the task-specific inference signal. The NLI results thus highlight the limit of how far one can push parameter-free upper FFNs before accuracy drops become unacceptable for a given application. At the same time, the overall picture shows that, for a broad class of classification tasks, it is possible to retain a large fraction of baseline performance while substantially lowering fine-tuning compute, which constitutes a favorable operating point for many real-world systems.

6. Discussion

Replacing the entire FFN in upper layers with a parameter-free orthogonal module—Q for the up-projection and $Q^\top$ for the down-projection—offers a simple way to reduce fine-tuning cost. On RoBERTa and BERT, the *total fine-tuning wall-clock time* (training plus evaluation) decreases by 40–50% on AGNews, XNLI and MNLI. On RoBERTa–IMDB, the reduction is smaller but remains clearly positive, which is consistent with the higher attention and evaluation cost for long documents [25]. IO-aware attention kernels and optimizer sharding can be applied in addition to this modification, further reducing overhead [26, 45]. For users operating under strict compute or energy budgets, such as on edge devices or shared accelerators, these savings can be more important than closing the remaining accuracy gap.

In terms of accuracy, IMDB and AGNews remain close to the baseline (within about one percentage point), whereas XNLI and MNLI show a larger decrease of around three points. This is consistent with evidence that upper layers encode task-specific semantics and inference signals, while lower layers focus more on general linguistic features [17, 18]. Under this view, keeping lower FFNs fully trainable and simplifying upper FFNs to fixed orthogonal modules is a natural compromise: feature extraction remains flexible, while task-specific mixing is approximated by a rich but parameter-free nonlinear mapping.

6.1. Significance and practicality

The proposed module is straightforward to implement in existing code. For each selected layer, a single orthogonal matrix Q and its transpose $Q^\top$ are generated, orthogonalized and frozen [15, 31]. Backpropagation passes through these mappings to earlier layers, but no FFN gradients or optimizer states are maintained for the replaced sublayers. The resulting savings in compute and parameter-update memory complement mixed-precision training and checkpointing [47, 48]. In distributed or memory-constrained regimes, the approach integrates well with ZeRO-style optimizers and fast attention kernels [26, 45]. Compared with methods that add new trainable modules, this design keeps the parameter set minimal and the optimizer footprint small, which simplifies deployment in constrained environments.

Orthogonality also plays a stabilizing role. It supports well-conditioned signal propagation in deep stacks, which matches both theoretical analyses and the ablation comparing orthogonal and non-orthogonal random replacements [29, 30]. In practice, the experiments show low variance across random seeds and consistent gains of orthogonal over purely random mappings.

6.2. Trade-offs and possible extensions

The main limitation appears on NLI, where accuracy drops are more pronounced than on single-sentence classification. This is expected: NLI often hinges on subtle semantic relations and cross-sentence reasoning, for which upper-layer capacity is likely to matter. There are several straightforward extensions on the modeling side, such as replacing only a subset of upper layers or partially unfreezing certain projections, rather than freezing all FFNs above a fixed cut. Another direction is to combine the proposed FFN replacement with sparse or linear attention variants and long-context architectures [6, 7, 25]. Such combinations would reduce both attention-side and FFN-side training cost, particularly for larger models and longer sequences.

This work focuses on the setting where a clear and predictable trade-off is desired: keep accuracy above approximately 95% of the baseline on a diverse set of tasks, while removing a substantial portion of FFN-related gradients and optimizer state. Reintroducing additional trainable components in the modified layers would move the method away from this goal, bringing back complexity in both implementation and optimization. For applications

where compute, memory or energy are the primary constraints, this type of simple reduction in training cost, with well-characterized accuracy behavior, can be preferable to more complex schemes that push accuracy slightly higher at the expense of additional overhead.

6.3. Broader impact

Reducing the cost of fine-tuning without introducing new trainable parameters can ease deployment on constrained hardware and lower the barrier to using large pretrained encoders. This is particularly relevant in settings where models must be adapted frequently or where many fine-tuning jobs share limited resources. Examples include user-facing services that continuously refine models based on feedback, decision-support tools in domains such as healthcare or finance, and platforms that host numerous task-specific variants of a backbone model. Edge and embedded scenarios, where memory and power budgets are tight and models must often be updated on-device, can particularly benefit from a method that offers predictable accuracy retention with significantly reduced compute [49]. In such contexts, trading a small and well-characterized amount of accuracy for a substantial reduction in fine-tuning time and resource usage can be an attractive option.

7. Conclusion

This work explored a simple strategy for accelerating Transformer fine-tuning by replacing the FFN sublayers in the upper half of encoder blocks with a parameter-free orthogonal module of the form $Q^\top \sigma(Qx)$. Across IMDB, AGNews, XNLI, and MNLI, the proposed modification preserves more than 95% of the baseline accuracy on average while reducing total fine-tuning wall-clock time (training plus evaluation) by 40–50%. These results suggest that a carefully chosen, fixed orthogonal replacement can remove a substantial portion of FFN-related training cost without altering the overall model interface or the standard fine-tuning workflow.

Beyond end-to-end speedups, the approach is practically lightweight: it leaves attention, residual connections, and normalization unchanged and does not introduce additional trainable parameters. Empirically, accuracy remains close to the baseline on single-sentence classification, while the degradation is more noticeable on NLI tasks, indicating that some inference-heavy settings benefit more from retaining trainable capacity in upper layers. Overall, the method provides a clear and reproducible operating point where a modest and predictable accuracy reduction can be traded for a substantial reduction in fine-tuning time.

Several directions remain open. A theoretical characterization of when $Q^\top \sigma(Qx)$ can safely approximate a learned FFN, and how the induced perturbation interacts with depth and residual pathways, would help predict which tasks and architectures are most suitable. On the empirical side, more systematic sweeps over the fraction and placement of replaced layers, the intermediate dimension, and partial replacements (e.g., freezing only one projection) would better map the speed–accuracy frontier. Finally, evaluating the same idea on larger backbones, decoder-only and encoder–decoder architectures, and long-context settings would clarify scalability and practical limits, and would help position this approach as a general component in compute-efficient Transformer adaptation.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, in: Advances in Neural Information Processing Systems, 2020.

[2] P. Michel, O. Levy, G. Neubig, Are sixteen heads really better than one?, in: Advances in Neural Information Processing Systems, 2019, pp. 14014–14024.

[3] V. Sanh, T. Wolf, A. M. Rush, Movement pruning: Adaptive sparsity by fine-tuning, in: Advances in Neural Information Processing Systems, 2020.

[4] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Tang, M. Jiang, S. Li, Z. Xu, Y. Wang, C. Xu, Q-BERT: Hessian based ultra low precision quantization of BERT, in: Proceedings of AAAI, Vol. 34, 2020, pp. 8815–8821.

[5] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, 8-bit optimizers via block-wise quantization, in: Advances in Neural Information Processing Systems, 2022.

[6] Y. Tay, M. Dehghani, D. Bahri, D. Metzler, Efficient transformers: A survey, ACM Computing Surveys 55 (6) (2022) 1–28.

[7] N. Kitaev, Ł. Kaiser, A. Levskaya, Reformer: The efficient transformer, in: International Conference on Learning Representations, 2020.

[8] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontañón, P. Pham, A. Ravula, Q. Wang, L. Yang, A. Ahmed, Big Bird: Transformers for longer sequences, in: Advances in Neural Information Processing Systems, 2020.

[9] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. D. Laroussilhe, A. Gesmundo, M. Attariyan, S. Gelly, Parameter-efficient transfer learning for NLP, in: International Conference on Machine Learning, 2019.

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations, 2022.

[11] P. Shaw, J. Uszkoreit, A. Vaswani, Self-attention with relative position representations, in: Proceedings of NAACL-HLT, 2018, pp. 464–468.

[12] B. Zhang, R. Sennrich, Root mean square layer normalization, in: Advances in Neural Information Processing Systems, 2019.

[13] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: International Conference on Learning Representations, 2015.

[14] N. Shazeer, M. Stern, Adafactor: Adaptive learning rates with sublinear memory cost, in: International Conference on Machine Learning, Vol. 80, PMLR, 2018, pp. 4596–4604.

[15] G. Huang, Q. Zhu, C. Siew, Extreme learning machine: Theory and applications, Neurocomputing 70 (1-3) (2006) 489–501.

[16] W. Cai, W. Zhang, Y. Zhang, Y. Wang, X. Chen, W. He, ELM-AE: Extreme learning machine autoencoder and its application in spectral classification, Sensors 18 (10) (2018) 3403.

[17] O. Kovaleva, A. Romanov, A. Rogers, A. Rumshisky, Revealing the dark secrets of BERT, in: Proceedings of EMNLP, 2019, pp. 4365–4374.

[18] A. Merchant, A. Rogers, A. Rumshisky, What happens to BERT embeddings during fine-tuning?, in: BlackboxNLP Workshop, 2020, pp. 33–44.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.

[20] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NAACL-HLT, 2019, pp. 4171–4186.

[21] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: A robustly optimized BERT pretraining approach, Tech. rep., Facebook AI, technical report (2019).

[22] S. Hao, S. Ren, C. Hou, P. Yin, G. Neubig, Y. Yang, How does BERT answer questions? a layer-wise analysis of transformer representations, in: Proceedings of COLING, 2020, pp. 5823–5835.

[23] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, D. Zhou, MobileBERT: A compact task-agnostic BERT for resource-limited devices, in: Proceedings of ACL, 2020.

[24] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, Q. Liu, TinyBERT: Distilling BERT for natural language understanding, in: Findings of EMNLP, 2020, pp. 4163–4174.

[25] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, D. Metzler, Long range arena: A benchmark for efficient transformers, in: International Conference on Learning Representations, 2021.

[26] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He, ZeRO: Memory optimizations toward training trillion parameter models, in: Proceedings of SC20, 2020.

[27] X. L. Li, P. Liang, Prefix-tuning: Optimizing continuous prompts for generation, in: Proceedings of ACL, 2021, pp. 4582–4597.

[28] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient fine-tuning of quantized LLMs, in: Advances in Neural Information Processing Systems, 2023.

[29] A. M. Saxe, J. L. McClelland, S. Ganguli, Exact solutions to the nonlinear dynamics of learning in deep linear neural networks, in: International Conference on Learning Representations, 2014.

[30] N. Bansal, X. Chen, Z. Wang, Can we gain more from orthogonality regularizations in training deep networks?, in: Advances in Neural Information Processing Systems, 2018, pp. 4261–4271.

[31] A. Rahimi, B. Recht, Random features for large-scale kernel machines, in: Advances in Neural Information Processing Systems, 2007, pp. 1177–1184.

[32] S. Wang, X. Wang, G. Huang, Dimension reduction with extreme learning machine, IEEE Transactions on Cybernetics 48 (2) (2018) 655–667.

[33] S. Yu, K. Jia, H. Lu, Learning deep representations via extreme learning machines, Neurocomputing 149 (2015) 308–315.

[34] Y. Yu, Q. Wang, Y. Wu, et al., Structured orthogonal random features based on discrete cosine transform, Neurocomputing 610 (2024) 128640.

[35] D. Hendrycks, K. Gimpel, Gaussian error linear units (GELUs), Tech. rep., University of California, Berkeley, technical report (2016).

[36] Y. N. Dauphin, A. Fan, M. Auli, D. Grangier, Language modeling with gated convolutional networks, in: International Conference on Machine Learning, 2017.

[37] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, T.-Y. Liu, On layer normalization in the transformer architecture, in: International Conference on Machine Learning, 2020, pp. 10524–10533.

[38] I. Loschilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations, 2019.

[39] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, C. Potts, Learning word vectors for sentiment analysis, in: Proceedings of ACL-HLT, 2011, pp. 142–150.

[40] X. Zhang, J. Zhao, Y. LeCun, Character-level convolutional networks for text classification, in: Advances in Neural Information Processing Systems, 2015.

[41] A. Williams, N. Nangia, S. Bowman, A broad-coverage challenge corpus for sentence understanding through inference, in: Proceedings of NAACL-HLT, 2018, pp. 1112–1122.

[42] A. Conneau, G. Lample, R. Rinott, A. Williams, S. R. Bowman, H. Schwenk, V. Stoyanov, XNLI: Evaluating cross-lingual sentence representations, in: Proceedings of EMNLP, 2018, pp. 2475–2485.

[43] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Brew, Transformers: State-of-the-art natural language processing, in: EMNLP 2020: System Demonstrations, 2020, pp. 38–45.

[44] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, 2019, pp. 8024–8035.

[45] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré, Flashattention: Fast and memory-efficient exact attention with I/O-awareness, in: Advances in Neural Information Processing Systems, 2022.

[46] T. Dao, H. Phan, A. Rudra, C. Ré, Flashattention-2: Faster attention with better parallelism and work partitioning, in: International Conference on Learning Representations, 2024.

[47] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, H. Wu, Mixed precision training, in: International Conference on Learning Representations, 2018.

[48] T. Chen, B. Xu, C. Zhang, C. Guestrin, Training deep nets with sublinear memory cost, Tech. rep., University of Washington, technical report (2016).

[49] A. Saha, S. Hadou, D. Samanta, Vision transformers on the edge: A comprehensive review of models, techniques, and the future, Neurocomputing 643 (2025) 130417.